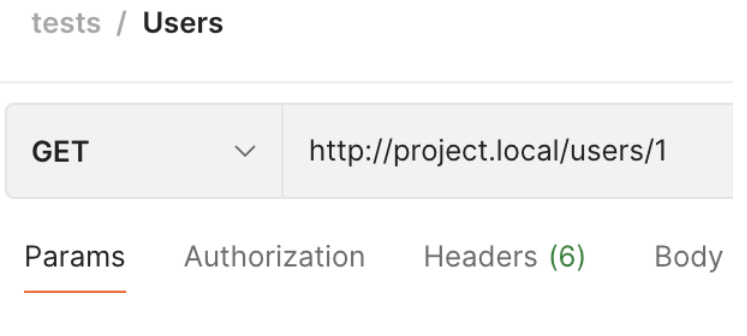
Введение в тестирование

1. Ручное тестирование в Postman

- Повторять проверки после изменений
- Время тратится на рутину



2. Автоматизация

```
resp := httptest.NewRequest("GET", "/users/1", nil)

assert.Equal(t, http.StatusOK, resp.StatusCode)
assert.Contains(t, resp.Header.Get("Content-Type"), "application/json")
```

3. Находить неудачные решения

Пример простой функции:

```
func Calculate(a, b int) int {
    return a + b
}
```

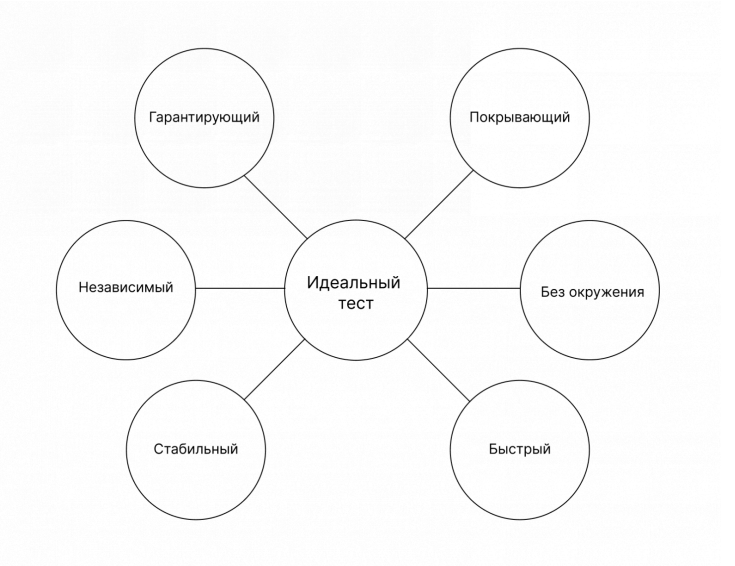
Тест на граничные значения:

```
result := Calculate(math.MaxInt64, 1) // -9223372036854775808
```

4. Не повторять старые баги

Идеальный тест ✨

- 🎯 **Гарантирует** работоспособность
- 🚀 **Быстро** выполняется
- 🏗️ **Не требует** окружения
- 💪 **Стабильный** результат
- 🔧 **Независим** от рефакторинга


Гарантирующий
Покрывающий
Независимый
Идеальный тест
Без окружения
Стабильный
Быстрый

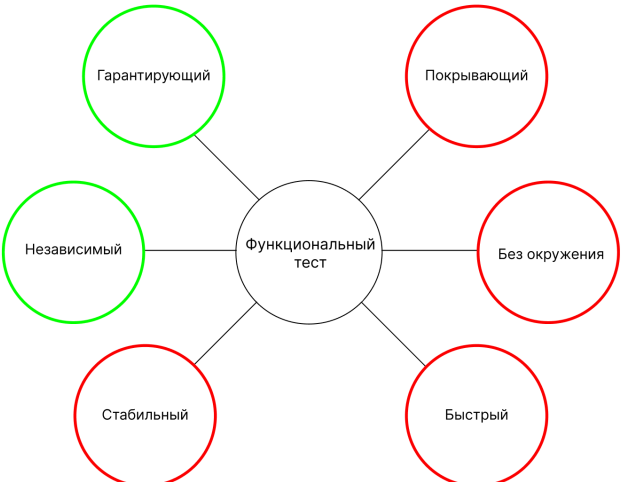
Виды тестов 📊

Функциональные тесты 🌐

Что тестируем: Весь функционал целиком

Особенности:

- ✅ Реальная инфраструктура
- ✅ Гарантия работы всей системы
- ❌ Медленные и нестабильные
- ❌ Требуют окружение

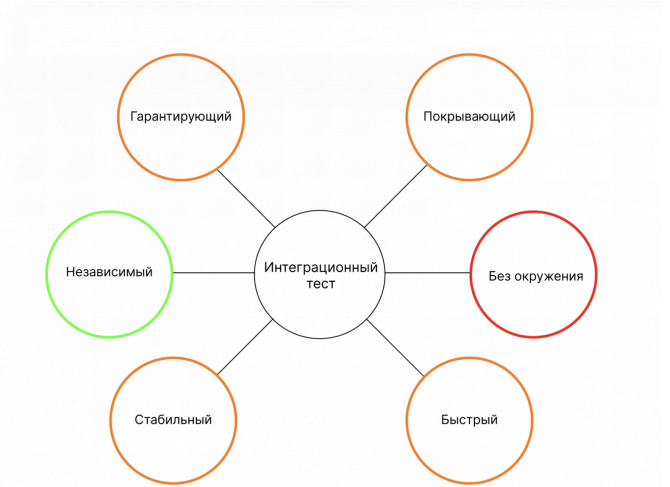

Гарантирующий
Покрывающий
Независимый
Функциональный тест
Без окружения
Стабильный
Быстрый

Интеграционные тесты 🔗

Что тестируем: Взаимодействие с инфраструктурой

Особенности:

- ✅ Проверка интеграций
- ✅ Больше тест-кейсов
- ✅ Быстрее функциональных
- ❌ Нужна база данных

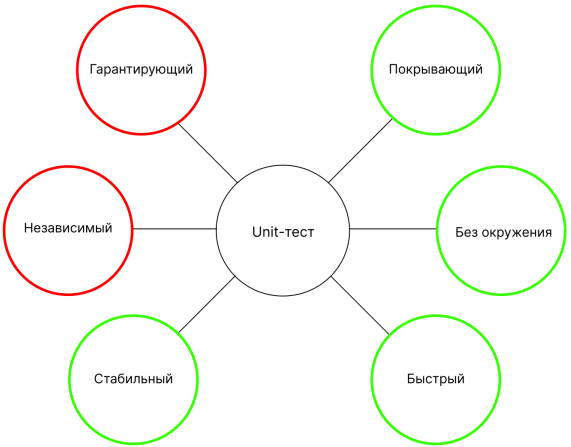

Гарантирующий
Покрывающий
Независимый
Интеграционный тест
Без окружения
Стабильный
Быстрый

Unit-тесты ⚡

Что тестируем: Бизнес-логику изолированно

Особенности:

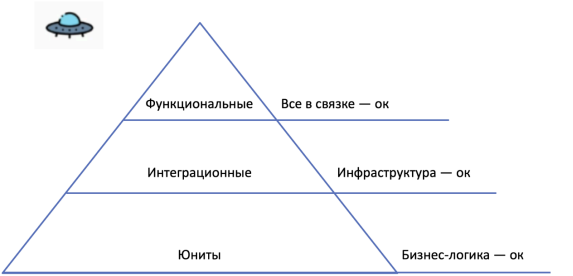
- ✅ Максимальное покрытие
- ✅ Самые быстрые
- ✅ Не нужно окружение
- ❌ Привязка к реализации


Гарантирующий
Покрывающий
Независимый
Unit-тест
Без окружения
Стабильный
Быстрый

Сравнение тестов 📋

	Функциональный	Интеграционный	Юнит
Гарантирующий			
Покрывающий			
Без окружения			
Быстрый			
Стабильный			
Независимый			

Пирамида тестирования



]

Принцип:

- 📍 **Много Unit** - быстро, дешево
- 📍 **Средне Integration** - важные интеграции
- 📍 **Мало Functional** - ключевые сценарии

Оптимизация CI/CD

Последовательная стратегия:

1. **Unit тесты** → Быстро, параллельно
2. **Integration тесты** → С тестовым окружением
3. **Functional тесты** → Полная проверка

Результат: Быстрая обратная связь

Лучшие практики 💡

Unit-тесты:

- Покрывать всю бизнес-логику
- Использовать контейнер зависимостей

Integration-тесты:

- Все инфраструктурные компоненты
- Mock-серверы для HTTP API

Практические советы 🛠️

Для HTTP-клиентов:

- Используйте Mock-серверы
- Не ходите в реальные API в тестах

Для рефакторинга:

- Настраивайте тестовое окружение
- Минимизируйте зависимость от реализации

Заключение 🎯

Главные выводы:

1. 🔄 **Комбинируйте** все виды тестов
2. 📐 **Следуйте** пирамиде тестирования
3. 🏗️ **Хороший код** → хорошие тесты
4. ⚡ **Быстрые тесты** → быстрая разработка

Помните: Если компонент сложно тестировать - проблема в архитектуре!

Спасибо за внимание! 👏